LIBROWARE

Library Management System

Congo-Cameroon Interstate University

Requirements Specification Document

Version 1.0

Date: May 13, 2026

Status: Draft — For Review

Prepared by: Libroware Engineering Team

Intended for: Congo-Cameroon Interstate University — Library Administration

Table des matières

Document Control

Version	Date	Author	Description
1.0	2026-05-13	Engineering Team	Initial draft

Definitions & Acronyms

Term / Acronym	Definition
RSD	Requirements Specification Document
API	Application Programming Interface
GraphQL	Query language and runtime for APIs, developed by Meta
JWT	JSON Web Token — a compact, URL-safe means of representing claims between two parties
ORM	Object-Relational Mapping — abstraction layer between code and database
CRUD	Create, Read, Update, Delete — the four basic data operations
CI/CD	Continuous Integration / Continuous Delivery — automated build and deployment pipeline
VPS	Virtual Private Server
FCFA	Franc CFA — currency used in Central African member states
CCISU	Congo-Cameroon Interstate University
ISBN	International Standard Book Number
UI/UX	User Interface / User Experience
SPA	Single Page Application
CORS	Cross-Origin Resource Sharing
HTTPS	Hypertext Transfer Protocol Secure

1. Introduction

1.1 Purpose of This Document

This Requirements Specification Document (RSD) formally describes the functional and non-functional requirements, system architecture, actors, and use cases of Libroware; a full-stack digital library management system developed for the Congo-Cameroon Interstate University (CCISU). It constitutes the authoritative technical and functional reference for all parties involved in the procurement, deployment, and maintenance of the system.

1.2 Project Background

The Congo-Cameroon Interstate University operates a physical library shared between faculties across its campuses. Until now, library operations; including book cataloguing, member registration, borrow tracking, and overdue management, have been conducted manually or through rudimentary spreadsheet tools. This approach is error-prone, difficult to audit, and unable to scale with the university's growing student body and book inventory.

Libroware was commissioned to replace this fragmented workflow with a centralised, web-based platform that is accessible from any device on the university network, offers role-based access control, and provides real-time visibility into the library's inventory and borrowing activity.

1.3 Scope

Libroware covers the following domains within CCISU's library operations:

- Digital cataloguing of books, authors, and categories
- Member registration and role-based access control (User, Librarian, Administrator)
- Borrow, return, and overdue tracking with status management
- Book review and rating by authenticated library members
- Profile and cover image management via cloud storage
- Administrative dashboards with key performance indicators
- Secure REST-over-GraphQL API consumed by the web frontend
- Automated CI/CD deployment to a Linux VPS via GitHub Actions

The following are explicitly out of scope for version 1.0:

- Integration with the university's student information system (planned in U-10)
- A native mobile application (planned in U-12)
- Financial fine management (planned in U-02)
- Email notification system (planned in U-01)

2. Motivation & Problem Statement

2.1 Current Situation

Prior to Libroware, the CCISU library operated without a dedicated digital system. Librarians maintained physical ledgers and Excel sheets to track borrowed books. This model presented several recurring challenges:

Problem	Impact
No centralised book catalogue	Librarians cannot quickly verify availability; students waste time searching manually
Manual borrow tracking	Overdue books go unnoticed; return rates are difficult to enforce
No member accounts	No borrowing history per student; identity verification is manual
No overdue alerts	Books remain unreturned for extended periods, reducing availability
No performance visibility	Management cannot report on borrowing trends, popular titles, or inventory gaps
Single point of failure	Loss of the physical register means loss of all records

2.2 Proposed Solution

Libroware addresses each of the above problems through a purpose-built, cloud-deployable web application. The system provides:

- **Centralised digital catalogue:** all books, authors, and categories managed in a single PostgreSQL database with full-text search.
- **Real-time borrow tracking:** each borrow record captures the borrower, book, due date, and status (BORROWED / RETURNED / OVERDUE), updated atomically.
- **Role-based access:** three distinct actor roles ensure that students cannot perform administrative actions, and that librarians operate within their authorised scope.
- **Audit trail:** every create, update, and delete operation on critical records is logged to an immutable audit log with the actor's identity and timestamp.
- **Soft deletes:** user and book records are never permanently lost; deleted entries are hidden from normal queries but recoverable by an administrator.
- **Secure authentication:** JWT-based sessions with configurable expiry, password hashing with bcrypt (cost factor 10), and rate-limited login endpoints.

3. Stakeholders & Actors

Libroware defines three authenticated actor roles. Every user of the system belongs to exactly one role, assigned at account creation. The role hierarchy is: Administrator>Librarian>User.

3.1 Administrator (ADMIN)

The Administrator holds the highest privilege level. This role is reserved for the Head Librarian or IT Manager of the CCISU library. There should be at most one or two Administrator accounts in a production deployment.

- Full CRUD access to all users, books, authors, and categories
- Create and deactivate Librarian accounts
- View the complete audit log
- Restore soft-deleted users and books
- Configure system-level settings (fine rates, borrow limits)
- Access all administrative dashboards and reports
- Override or waive fines (planned, U-02)

3.2 Librarian (LIBRARIAN)

Librarians are the day-to-day operators of the system. They are staff members employed by the CCISU library. Their accounts are created by an Administrator.

- Create, update, and delete book records (including cover image upload)
- Create and update author and category records
- Register new library members (USER accounts)
- Process borrow and return transactions on behalf of members
- View all active, overdue, and returned borrows
- View member profiles and borrowing history
- Cannot delete users or access the audit log

3.3 Library Member (USER)

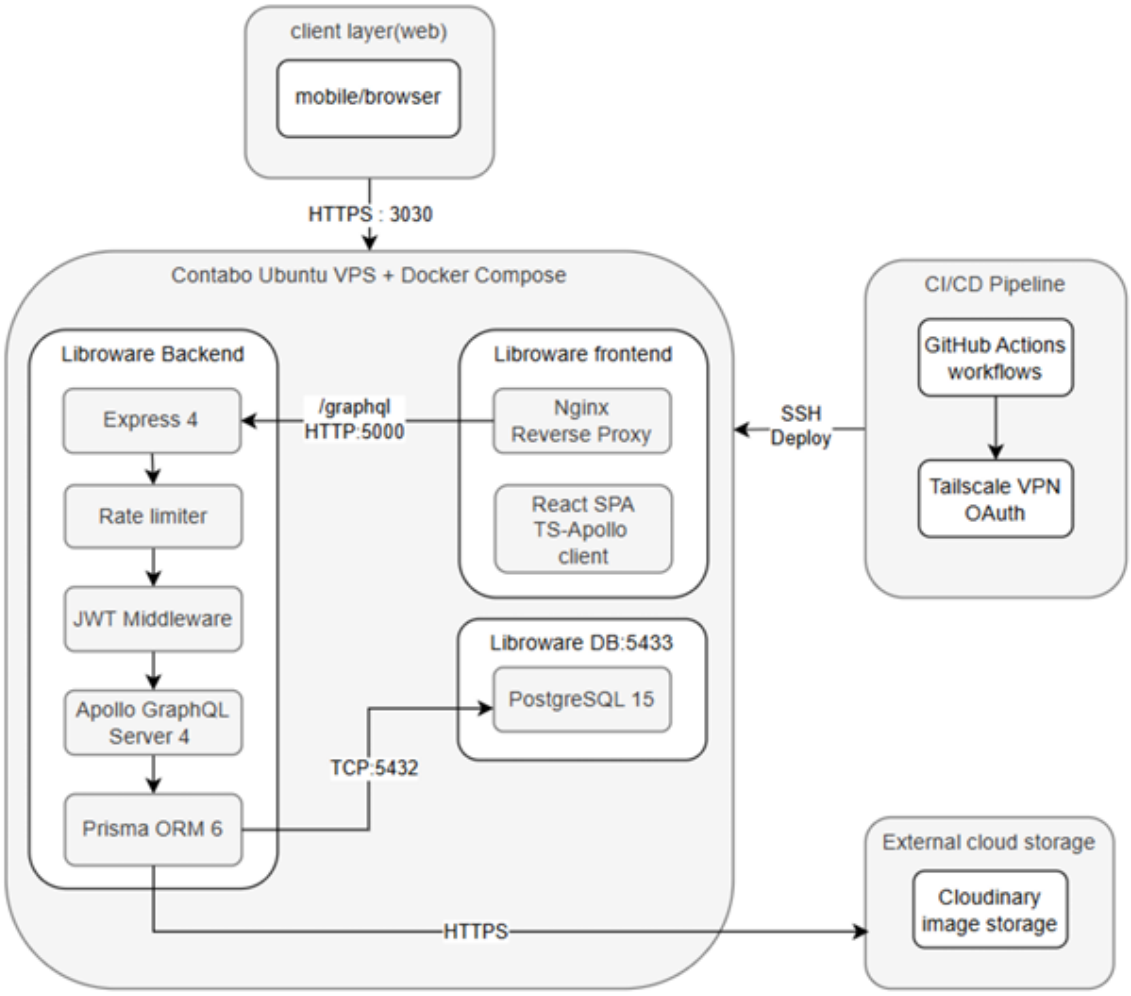
Library Members are students, faculty, or staff of the CCISU who have been registered in the system by a Librarian or Administrator. This is the default role assigned on account creation.

- Browse and search the book catalogue
- View their own active and past borrows
- Submit, edit, and delete their own book reviews and ratings
- Update their own profile (name, email, profile picture, password)
- Cannot access administrative panels or other users' data

4. System Architecture

4.1 Architectural Overview

Libroware follows a three-tier architecture: a React single-page application (SPA) as the presentation layer, a Node.js/Express GraphQL API as the application layer, and a PostgreSQL relational database as the data layer. All components are containerised with Docker and orchestrated via Docker Compose on a Linux VPS.



4.2 Frontend Architecture

The frontend is a React 18 Single Page Application written in TypeScript. It is built with Vite and styled with Tailwind CSS. Apollo Client manages all GraphQL communication and provides a normalised in-memory cache that eliminates redundant network requests.

Concern	Technology / Approach
Framework	React 18 with TypeScript
Build toolchain	Vite 5 — fast HMR in development, Rollup-based production build
Styling	Tailwind CSS 3 — utility-first; PostCSS with autoprefixer and cssnano
State & data	Apollo Client 3 — normalised cache, optimistic UI, error links
Routing	React Router DOM 6 — declarative, nested route definitions
Authentication	JWT stored in localStorage; AuthContext provides reactive auth state
Image handling	browser-image-compression before upload; Cloudinary for CDN delivery
Charts	Recharts + Chart.js for dashboard visualisations
Internationalisation	i18next (installed; localisation pending U-09)
Static serving	Nginx Alpine inside Docker container

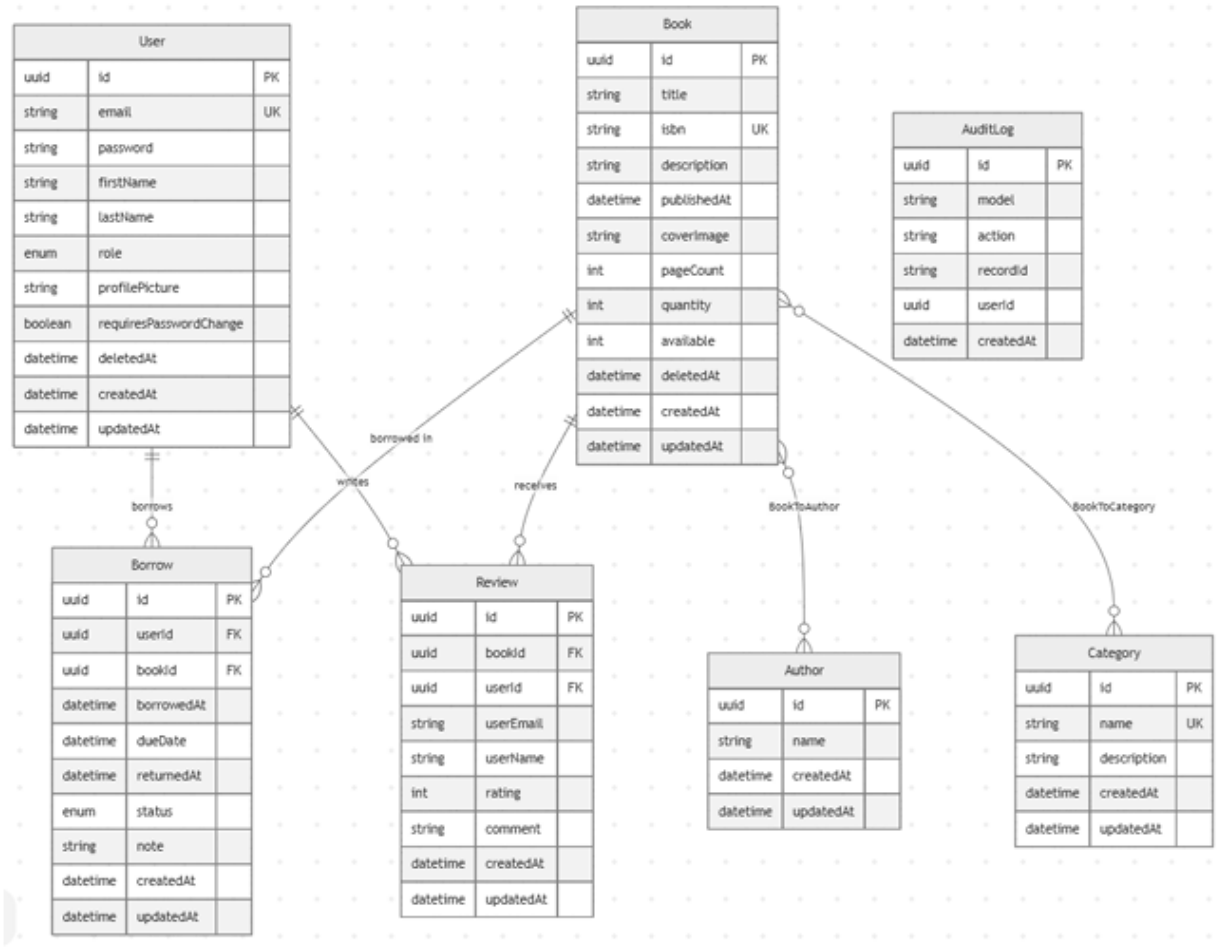
4.3 Backend Architecture

The backend is a Node.js application built on Express and Apollo Server 4. It exposes a single GraphQL endpoint at /graphql, secured by JWT authentication middleware and rate limiting. A single PrismaClient instance is instantiated at startup and passed through Apollo context to every resolver, ensuring connection pool efficiency.

Concern	Technology / Approach
Runtime	Node.js 20 LTS
HTTP framework	Express 4
API layer	Apollo Server 4 (GraphQL) — single /graphql endpoint
ORM	Prisma 6 — type-safe database access, migration engine
Authentication	jsonwebtoken (JWT) — HS256, 7-day expiry
Password hashing	bcryptjs — cost factor 10
Rate limiting	express-rate-limit — 100 req/15 min global; 10 req/15 min on auth mutations
CORS	Allowlist-based via ALLOWED_ORIGINS environment variable
Image storage	Cloudinary SDK — reads CLOUDINARY_URL env var; automatic transformation
Audit logging	Prisma \$extends query extension + AsyncLocalStorage for actor identity
Process manager	Docker (CMD node src/index.js); nodemon in development

4.4 Database Schema

All data is persisted in PostgreSQL 15. Prisma manages schema migrations. The following entity-relationship summary describes the core models:



4.5 Infrastructure & Deployment

All services are containerised using Docker and orchestrated with Docker Compose. The production environment runs on a Linux VPS (Ubuntu 22.04) hosted on Contabo. A GitHub Actions workflow automates the full build and deployment pipeline on every push to the main branch, connecting to the server via Tailscale VPN for private deployments.

Service	Container	Exposed Port	Image
Frontend (React + Nginx)	libroware-frontend	3030 → 80	node:20-slim + nginx:alpine
Backend (Node.js API)	libroware-backend	5000 → 5000	node:20-slim
Database (PostgreSQL)	libroware-postgres	5433 → 5432	postgres:15-alpine

5. Functional Requirements

5.1 Authentication & Access Control

ID	Requirement	Actor
FR-01	The system shall allow any visitor to sign up with email, password, first name, and last name.	All
FR-02	The system shall validate email format and enforce a minimum password length of 8 characters on signup and password change.	All
FR-03	The system shall authenticate users via email/password and return a JWT valid for 7 days.	All
FR-04	First-time accounts created by an Admin shall require the user to set a new password before accessing the system.	User, Librarian
FR-05	The system shall limit authentication attempts to 10 requests per 15 minutes per IP address.	System
FR-06	The system shall reject requests with expired or tampered JWT tokens.	System
FR-07	Role assignment shall be enforced server-side; no client input shall override a user's role.	System

5.2 Book Catalogue Management

ID	Requirement	Actor
FR-08	Authorised staff shall create book records with title, ISBN, description, publication date, cover image, page count, and quantity.	Admin, Librarian
FR-09	The system shall reject duplicate ISBNs.	System
FR-10	Staff shall update any field of an existing book record.	Admin, Librarian
FR-11	Staff shall soft-delete a book; soft-deleted books shall not appear in catalogue queries.	Admin, Librarian
FR-12	The system shall prevent deletion of books that have active borrows (status = BORROWED).	System
FR-13	Users shall browse all books with pagination (skip/take) and filter by title, author, or category.	All (authenticated)
FR-14	Book cover images shall be uploaded to Cloudinary and served via CDN URL.	Admin, Librarian
FR-15	The system shall track available copies separately from total quantity, decrementing on borrow and incrementing on return.	System

5.3 Author & Category Management

ID	Requirement	Actor
FR-16	Staff shall create, update, and delete authors.	Admin, Librarian
FR-17	Staff shall create, update, and delete categories; category names shall be unique.	Admin, Librarian
FR-18	Books shall be associated with one or more authors and one or more categories at creation or via update.	Admin, Librarian

5.4 Borrow & Return Management

ID	Requirement	Actor
FR-19	Staff shall create a borrow record specifying the member, book, due date, and optional note.	Admin, Librarian
FR-20	The system shall reject borrow creation if the due date is not in the future.	System
FR-21	The system shall atomically verify availability and decrement the available count within a single database transaction to prevent race conditions.	System
FR-22	Members shall view their own active and historical borrow records with status indicators.	User
FR-23	Staff shall view all borrows, filterable by status (BORROWED, RETURNED, OVERDUE).	Admin, Librarian
FR-24	Staff shall process a book return, setting status to RETURNED and recording returnedAt.	Admin, Librarian
FR-25	The system shall automatically transition borrow status from BORROWED to OVERDUE when dueDate has passed, using a bulk database update.	System

5.5 User Management

ID	Requirement	Actor
FR-26	Administrators shall create user accounts with any role (USER, LIBRARIAN, ADMIN).	Admin
FR-27	The system shall prevent non-admin users from assigning the ADMIN role to any account.	System
FR-28	Administrators shall soft-delete user accounts; soft-deleted users shall not appear in user listings.	Admin
FR-29	Users shall update their own profile: name, email, password, and profile picture.	User, Admin, Librarian
FR-30	Profile pictures shall be uploaded to Cloudinary, resized to 500×500 px.	All (authenticated)

5.6 Review & Rating System

ID	Requirement	Actor
FR-31	Authenticated users shall submit one review per book, with a rating from 1 to 5 and an optional comment.	User
FR-32	The system shall reject duplicate reviews from the same user for the same book.	System
FR-33	Review comments shall not exceed 2,000 characters.	System
FR-34	Users shall update or delete their own reviews; Administrators may delete any review.	User, Admin
FR-35	Reviews shall be cascade-deleted when the associated book or user is hard-deleted.	System

5.7 Audit Logging

ID	Requirement	Actor
FR-36	The system shall record every CREATE, UPDATE, and DELETE operation on User, Book, and Borrow models.	System
FR-37	Each audit log entry shall capture: model name, action type, affected record ID, actor user ID, and timestamp.	System
FR-38	Audit log entries shall be immutable and not exposed to USER or LIBRARIAN roles.	System

6. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	API responses for catalogue queries shall complete in under 500 ms under normal load (≤ 50 concurrent users).
NFR-02	Performance	The frontend SPA shall achieve a Lighthouse Performance score of ≥ 80 in production.
NFR-03	Security	All passwords shall be hashed using bcrypt with a work factor of 10 before storage; plaintext passwords shall never be persisted.
NFR-04	Security	CORS shall only allow requests from origins explicitly listed in the ALLOWED_ORIGINS environment variable.
NFR-05	Security	The server shall not start if the JWT_SECRET environment variable is absent.
NFR-06	Security	Rate limiting shall be applied to all authentication mutations (login, signup) at 10 requests per 15-minute window per IP.
NFR-07	Availability	The system shall target 99% uptime during university operating hours (07:00–22:00 UTC+1).
NFR-08	Scalability	The database connection pool shall be managed by a single Prisma instance; the architecture shall support horizontal scaling of the API tier via load balancer in future.
NFR-09	Maintainability	All configuration (database URL, JWT secret, Cloudinary credentials, allowed origins) shall be externalised to environment variables; no secrets shall be committed to source control.
NFR-10	Data Integrity	Soft deletes shall be used for User and Book records to preserve referential integrity and audit history; hard deletes shall be reserved for administrative recovery operations.
NFR-11	Usability	The UI shall be fully responsive and usable on desktop and tablet viewports (≥ 768 px width).
NFR-12	Compliance	The system shall not store any personally identifiable information beyond what is necessary for library membership (name, institutional email).

7. Key Use Cases

UC-01: Member borrows a book

Field	Detail
Actor	Library Member (User)
Preconditions	User is authenticated with appropriate role
Main Flow	1. The member is authenticated and browses the catalogue. 2. The member selects a book with available copies > 0. 3. A Librarian or Admin creates a borrow record with the member's ID and a future due date. 4. The system atomically verifies availability and decrements the available count. 5. A borrow record with status BORROWED is created and visible to the member.
Success Outcome	The borrow is created. Available count decrements by 1.
Alternative / Error	Book has zero available copies → system rejects with an error.

UC-02: Member returns a book

Field	Detail
Actor	Librarian / Administrator
Preconditions	User is authenticated with appropriate role
Main Flow	1. The librarian identifies the borrow record (by member name or borrow ID). 2. The librarian triggers the Return Book action. 3. The system sets status = RETURNED, records returnedAt, and increments available count.
Success Outcome	Borrow status changes to RETURNED. Book becomes available.
Alternative / Error	Borrow is already RETURNED → system rejects with an error.

UC-03: Administrator creates a Librarian account

Field	Detail
Actor	Administrator
Preconditions	User is authenticated with appropriate role
Main Flow	1. Admin navigates to User Management → Create User. 2. Admin enters name, institutional email, temporary password, and selects role LIBRARIAN. 3. System validates email format and password length. 4. Account is created with requiresPasswordChange = true. 5. Librarian logs in and is prompted to set a permanent password before proceeding.
Success Outcome	Librarian account is active with forced password reset on first login.
Alternative / Error	Email already exists → system rejects with a conflict error.

UC-04: System marks borrows as overdue

Field	Detail
Actor	System (automated)
Preconditions	User is authenticated with appropriate role
Main Flow	1. On each borrows query, the system executes a bulk updateMany operation. 2. All borrow records with status = BORROWED and dueDate < now() are transitioned to OVERDUE. 3. Updated records are returned in the query response.
Success Outcome	Overdue borrows are correctly reflected in all queries.
Alternative / Error	No eligible records → no update performed.

8. System Constraints & Assumptions

8.1 Constraints

- The system requires a stable internet connection for Cloudinary image upload/delivery. A fully offline mode is not supported in v1.0.
- All production environment variables must be configured on the server before deployment; the application will refuse to start if JWT_SECRET is absent.
- The current authentication model stores JWT tokens in browser localStorage, which is susceptible to XSS. Migration to httpOnly cookies is planned in upgrade U-05.
- PostgreSQL must be version 13 or higher due to Prisma 6 compatibility requirements.
- Node.js 20 LTS is required for both development and production environments.

8.2 Assumptions

- CCISU will provide at minimum one Linux VPS (Ubuntu 22.04+) with Docker and Docker Compose installed.
- The university's network allows outbound HTTPS traffic to GitHub (for CI/CD) and Cloudinary (for image storage).
- An institutional email domain will be used for all staff accounts; member email validation is format-only and does not verify domain.
- A Cloudinary account (free or paid tier) will be provisioned and the CLOUDINARY_URL credential provided to the engineering team for configuration.
- A single instance of the application serves all faculties of the university. Multi-tenancy is not required in v1.0.

9. Project Management Plan

The Libroware project is planned over a 12-week (3-month) delivery window. The work is organised into nine sequential phases, each producing a testable deliverable. The plan below details every task, the responsible profile, and the estimated duration.

9.1 Team Composition & Required Profiles

The recommended team for a 3-month delivery consists of seven roles. Part-time allocations are expressed as a percentage of a standard 160-hour working month.

Profile	Allocation	Key Skills Required
Technical Lead / Project Manager	Full-time	Software architecture, GraphQL API design, project planning, stakeholder communication, Node.js, PostgreSQL.
Senior Full-Stack Developer	Full-time	React 18, TypeScript, Node.js, Apollo GraphQL, Prisma ORM, Docker. Supports both frontend and backend tasks.
Frontend Developer	Full-time	React, TypeScript, Tailwind CSS, Apollo Client, responsive UI implementation.
Backend Developer	Full-time	Node.js, Express, GraphQL resolver design, PostgreSQL, Prisma, JWT authentication, security best practices.
UI/UX Designer	50% (part-time)	Figma or equivalent, design systems, accessibility principles, user research, wireframing.
DevOps / Infrastructure Engineer	30% (part-time)	Docker, Docker Compose, Linux server administration, GitHub Actions CI/CD, Nginx, VPN (Tailscale), SSH. Minimum 2 years of experience.
QA / Test Engineer	50% (part-time)	Manual and automated testing, API testing (Postman/Insomnia), test case design, security testing basics. Minimum 1 year of experience.

9.2 Work Breakdown Structure (WBS)

The project is divided into nine phases. Each task is assigned a unique identifier, a responsible profile, and a duration in working days.

ID	Task	Responsible	Duration	Weeks
	PHASE 1 — Foundation & Environment Setup			1–2
T-01	Requirements finalisation & technical kick-off	Tech Lead	3 days	1
T-02	Development environment, Git repository, Docker setup	DevOps	3 days	1
T-03	Database schema design, Prisma initialisation, first migration	Backend Dev	3 days	1–2
T-04	Express / Apollo GraphQL scaffolding, project structure	Backend Dev	2 days	2
T-05	CI/CD pipeline (GitHub Actions), staging deployment	DevOps	3 days	2
	PHASE 2 — Authentication & User Management			2–4
T-06	JWT authentication (login, signup, token expiry)	Backend Dev	3 days	2–3
T-07	Role-based access control (ADMIN, LIBRARIAN, USER)	Backend Dev	2 days	3
T-08	User CRUD resolvers (createUser, updateUser, deleteUser)	Backend Dev	3 days	3
T-09	Forced password change flow (requiresPasswordChange)	Full-Stack Dev	2 days	3–4
T-10	Login page & user setup form (frontend)	Frontend Dev	3 days	3–4
T-11	Profile picture upload to Cloudinary (frontend + backend)	Full-Stack Dev	2 days	4
	PHASE 3 — Book Catalogue			4–6
T-12	Author & Category CRUD (resolvers + GraphQL schema)	Backend Dev	2 days	4
T-13	Book CRUD with author/category associations	Backend Dev	3 days	4–5
T-14	Book cover image upload (Cloudinary)	Full-Stack Dev	2 days	5
T-15	Book search & filter (by title, author, category)	Backend Dev	2 days	5
T-16	Catalogue UI: book listing, search, detail view	Frontend Dev	4 days	5–6
T-17	Author & category management UI (admin)	Frontend Dev	2 days	6
	PHASE 4 — Borrow & Return System			6–8
T-18	Borrow creation (atomic transaction, availability check)	Backend Dev	3 days	6–7
T-19	Return processing (status update, available increment)	Backend Dev	2 days	7
T-20	Overdue status automation (bulk updateMany)	Backend Dev	2 days	7
T-21	Borrow management UI (admin: create, view, return)	Frontend Dev	3 days	7–8

T-22	Member borrow history UI (user dashboard)	Frontend Dev	2 days	8
	PHASE 5 — Review & Rating System			8–9
T-23	Review CRUD resolvers with rating validation	Backend Dev	2 days	8
T-24	Review UI on book detail page	Frontend Dev	2 days	8–9
	PHASE 6 — Admin Panel & Dashboard			8–10
T-25	Admin panel: user management tab	Full-Stack Dev	3 days	8–9
T-26	Admin panel: book management tab	Full-Stack Dev	2 days	9
T-27	Dashboard KPIs (total books, borrows, overdue, members)	Frontend Dev	3 days	9–10
T-28	Audit log backend (Prisma extension + AsyncLocalStorage)	Backend Dev	2 days	9
	PHASE 7 — Security Hardening			10–11
T-29	Rate limiting (global + auth-specific)	Backend Dev	1 day	10
T-30	CORS allowlist, environment variable validation	Backend Dev	1 day	10
T-31	Soft deletes (User, Book) + input validation audit	Backend Dev	2 days	10
T-32	UI/UX review pass — polish, consistency, accessibility	UI/UX Designer	4 days	10–11
	PHASE 8 — Testing & Quality Assurance			10–12
T-33	Test plan & test case writing	QA Engineer	3 days	10
T-34	API testing (all mutations and queries via Postman)	QA Engineer	4 days	10–11
T-35	Frontend regression testing (all user flows)	QA Engineer	3 days	11
T-36	Security audit (auth bypass, injection, CORS, rate limit)	Tech Lead + QA	3 days	11
T-37	Bug fixing sprint	Full-Stack Dev + Backend Dev	4 days	11–12
	PHASE 9 — Deployment, Training & Handover			12
T-38	Production deployment on CCISU VPS	DevOps	2 days	12
T-39	Database seeding (admin account, initial data)	Backend Dev	1 day	12
T-40	Staff training session (2 days, library staff)	Tech Lead	2 days	12
T-41	User Acceptance Testing (UAT) with library staff	QA + Tech Lead	2 days	12
T-42	Technical documentation & handover package	Tech Lead	2 days	12

9.3 Delivery Timeline Summary

Phase	Description	Key Deliverable	Weeks
1	Foundation & Environment	Running Docker stack, CI/CD pipeline, DB schema	1–2
2	Authentication & Users	Login, signup, RBAC, profile management	2–4
3	Book Catalogue	Book/author/category CRUD, search, cover images	4–6
4	Borrow & Return System	Full borrow lifecycle, overdue automation	6–8
5	Review & Rating	Book reviews with rating, moderation	8–9
6	Admin Panel & Dashboard	Admin UI, KPI dashboard, audit log	8–10
7	Security Hardening	Rate limiting, soft deletes, validation, CORS	10–11
8	Testing & QA	Test plan, API tests, regression, security audit	10–12
9	Deployment & Handover	Production deployment, staff training, UAT, documentation	12

Note: Phases 5, 6, and 7 overlap partially to maximise parallelism between frontend, backend, and design tasks. The Tech Lead reviews all pull requests and conducts weekly progress checkpoints with the CCISU library administration.

10. Financial Estimation

10.1 Infrastructure & Third-Party Services

Item	Type	Unit Cost (FCFA)	Qty / Period	Subtotal (FCFA)
Linux VPS — Contabo (4 vCPU, 8 GB RAM)	Annual	10,000	1 year	120,000
SSL certificate (Let's Encrypt)	Annual	0	1 year	0
Cloudinary (paid tier — 10 GB storage)	Annual	13,000	1 year	130,000
Subtotal — Infrastructure				250,000

10.2 One-Time & Miscellaneous Costs

Item	Description	Estimated Cost (FCFA)
Development tooling & software licences	Figma, JetBrains IDE, Postman Team plan	120,000
Technical documentation	User manual, admin guide, API reference (print + digital)	80,000
Subtotal — Miscellaneous		200,000

10.3 Total Project Budget

Budget Line	Amount (FCFA)
Infrastructure & Third-Party Services	250,000
One-Time & Miscellaneous Costs	200,000
TOTAL PROJECT COST	450,000

10.4 Annual Operational Cost (Post-Delivery)

After the initial 3-month delivery, the following recurring costs are required to keep the system operational.

Item	Frequency	Annual Cost (FCFA)
Linux VPS hosting	Monthly	120,000
Cloudinary storage	Monthly	130,000
Domain name renewal	Annual	10,000
TOTAL ANNUAL OPERATIONAL COST		165,000

NB: Human ressources payment have not been included in the financial estimations.

11. Planned Upgrades Summary

The following enhancements have been identified and formally documented in the accompanying UPGRADES.md file. They are not part of the v1.0 delivery but are included here for planning and budgeting purposes.

ID	Title	Priority	Impact
U-01	Email Notifications (due date & overdue reminders)	High	Reduces overdue rate significantly
U-02	Fine / Penalty System	High	Enforces accountability, enables revenue recovery
U-03	Book Reservation Queue	High	Improves availability fairness
U-04	Frontend Pagination	High	Correctness for large catalogues
U-05	JWT in httpOnly Cookies	High	Eliminates XSS token theft risk
U-06	Profile Update Without Page Reload	High	UX improvement
U-07	QR Code / Barcode Generation	Medium	Accelerates physical borrow workflow
U-08	Advanced Analytics Dashboard	Medium	Management decision support
U-09	Multi-Language Support (FR / EN)	Medium	Accessibility for bilingual campus
U-10	Student ID Integration	Medium	Single identity source of truth
U-11	Soft-Delete Restore UI	Medium	Operational recovery capability
U-12	Mobile Application (React Native)	Low	Extends reach to mobile-first students
U-13	Interlibrary Loan Module	Low	Cross-campus resource sharing
U-14	Audit Log Viewer in Admin Panel	Low	Compliance and accountability UI
U-15	Reading Progress Tracker	Low	Student engagement feature

Appendix A — GraphQL API Reference (Summary)

The complete API is available at the /graphql endpoint. The following tables summarise the principal queries and mutations.

Queries

Query	Arguments	Returns	Auth
me	—	User	Any
user	id: ID!	User	Any
users	skip, take	[User!]!	Admin, Librarian
book	id: ID!	Book	Any
books	skip, take, searchTitle	[Book!]!	Any
booksByAuthor	authorId, skip, take	[Book!]!	Any
booksByCategory	categoryId, skip, take	[Book!]!	Any
authors	skip, take, searchName	[Author!]!	Any
categories	skip, take	[Category!]!	Any
borrows	skip, take, status	[Borrow!]!	Admin, Librarian
userBorrows	userId, status, skip, take	[Borrow!]!	Any
overdueBorrows	—	[Borrow!]!	Admin, Librarian
bookReviews	bookId, skip, take	[Review!]!	Any

Mutations

Mutation	Key Inputs	Returns	Auth
signup	UserCreateInput	AuthPayload	Public
login	LoginInput	AuthPayload	Public
createUser	UserCreateInput	User	Admin
updateUser	id, UserUpdateInput	User	Owner / Admin
deleteUser	id	User	Owner / Admin
createBook	BookCreateInput	Book	Admin, Librarian
updateBook	id, BookUpdateInput	Book	Admin, Librarian
deleteBook	id	Book	Admin, Librarian
createAuthor	AuthorCreateInput	Author	Admin, Librarian
createCategory	CategoryCreateInput	Category	Admin, Librarian
createBorrow	BorrowCreateInput	Borrow	Admin, Librarian
returnBook	id	Borrow	Admin, Librarian
createReview	ReviewCreateInput	Review	User
updateReview	id, ReviewUpdateInput	Review	Owner / Admin
deleteReview	id	Review	Owner / Admin
uploadProfilePicture	userId, imageData	UploadResponse	Owner / Admin
uploadFile	file, type, id	UploadResponse	Admin, Librarian